pg_project 아키텍처 가이드

폐쇄망 PostgreSQL 설치 · HA · 백업 · 모니터링 · 튜닝 — 그림으로 보는 구성

naturalTuna/pg_project · RHEL 계열 8/9 (Rocky, Alma) x86_64 · 2026-11

목차

1. 한눈에 보기
2. 패키지 (pg_download.sh)
3. single 구성
4. HA 구성
5. HA 설치 흐름
6. 장애 시 동작
7. 안전장치 (split-brain / 유실 방지)
8. VIP 3중 관리
9. 백업 — basebackup vs pgBackRest
10. 노드 추가 / DR
11. 모니터링 — pgmon
12. 쿼리 튜닝 — pg_tune
13. 포트 · 디렉토리

같이 보기: 설치가이드(명령어, 체크리스트) · 운영가이드(일상 점검, 장애 대응) · pgmon_가이드

1. 한눈에 보기

인터넷이 되는 서버에서 설치에 필요한 모든 것을 `postgresql/` 한 디렉토리로 모으고, 그걸 폐쇄망으로 옮겨 **소스 빌드 → initdb → 튜닝 → 백업 → (HA) Patroni 구성**까지 스크립트 한 번으로 끝낸다. 운영은 pgmon(모니터링)과 pg_tune(튜닝)으로.

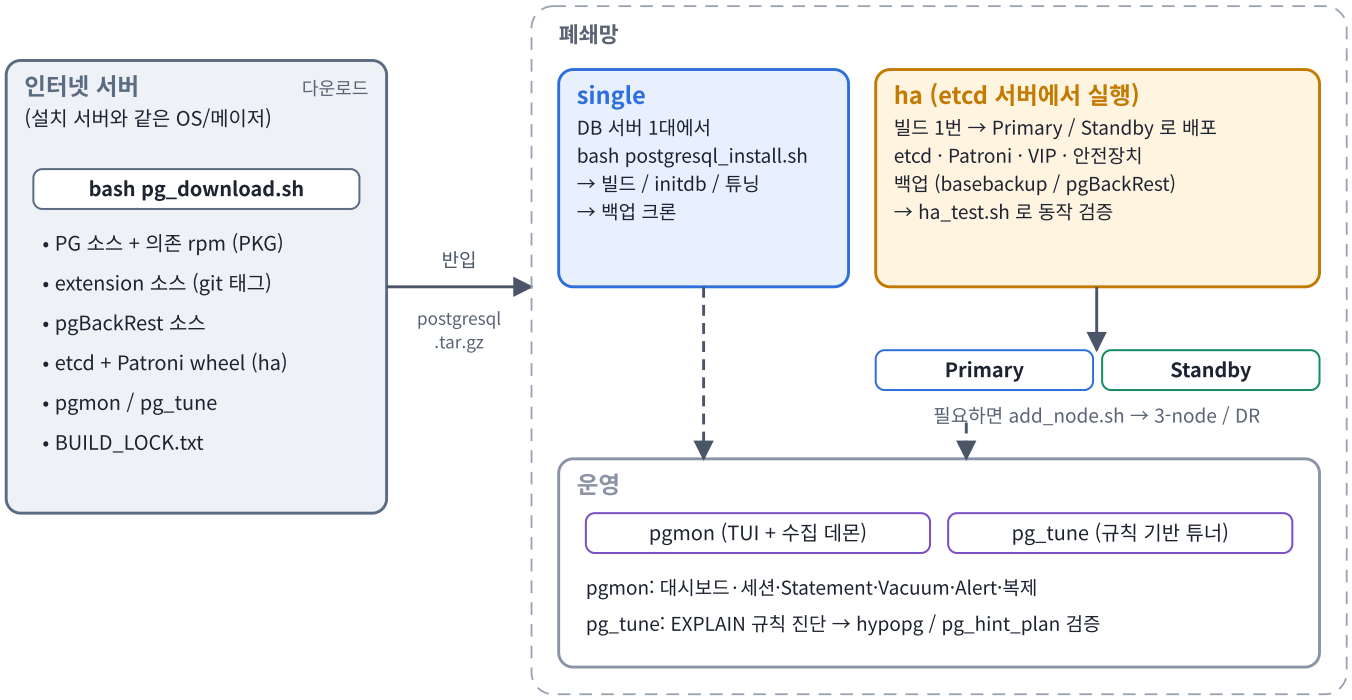


그림 1. 전체 흐름

2. 패키지 (pg_download.sh)

사람이 "PG 몇 버전엔 무슨 rpm" 표를 관리하지 않게, 받은 소스와 GitHub 태그를 보고 자동으로 정한다. 다운로드 서버는 **설치 서버와 같은 OS / 메이저 버전**이어야 한다 (rpm · python 버전).

postgresql/ (pg_download.sh 결과)

postgresql_install.sh lib/	설치 진입점 + 모듈
add_node.sh ha_test.sh	노드 추가 / HA 테스트
install.conf.sample	무인 설치 응답 파일
BUILD_LOCK.txt	구성 기록 (--lock 재현)
PKG/	의존 rpm
installer/postgresql-x.y.tar.gz	PG 소스
installer/build_features.txt	configure 옵션 (자동 판단)
installer/extensions/	hypopg · pg_hint_plan · pgvector
installer/pgbackrest/	pgBackRest 소스 + 휠
installer/etcd-*.tar.gz	etcd (ha)
installer/patroni_pkgs.tar.gz	Patroni wheel (ha)
monitoring/pgmon/	pgmon
tools/pg_tune/	pg_tune

자동으로 정하는 것

PG 버전	지원 버전 표에서 선택 (기본: 두 번째 메이저)
빌드 기능	받은 소스의 configure 로 지원 옵션만
	→ lz4 / zstd / uuid / systemd / io_uring(18+)
extension 태그	GitHub 최신 정식 릴리스 (ref_pattern)
	→ pg_hint_plan 은 메이저 전용 REL 태그
pgBackRest	최신 release/x.y.z
rpm	manifest.json + 기능별 -devel, 의존성까지
	→ CRB / PowerTools 자동 활성화
--verify	다운로드 서버에서 시험 빌드 (선택)
	→ 실패한 extension 은 이전 태그로
--lock	BUILD_LOCK.txt 와 같은 구성으로

그림 2. 패키지 구성과 자동 판단 항목

3. single 구성

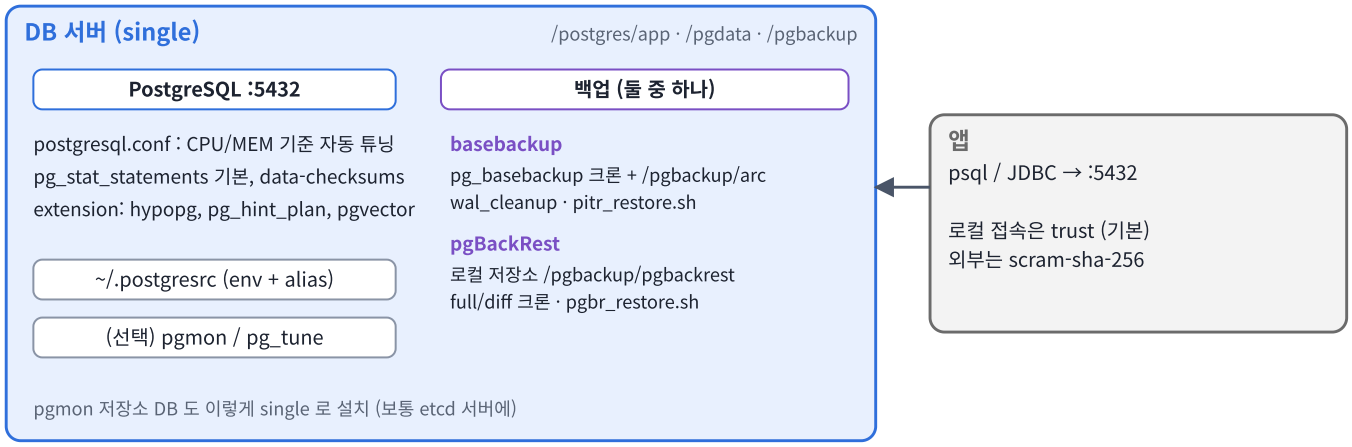


그림 3. single — DB 서버 1대

pgmon 저장소 DB 도 이 방식으로 설치한다. HA 라면 보통 etcd 서버에 single 로 하나 두고 pgmon 을 같이 돌린다.

4. HA 구성

3대: **etcd 서버**(설치 실행 + 감시 + 백업 저장소) · **Primary** · **Standby**. Patroni 가 PG 기동 / 승격 / 복제를 관리하고, etcd 의 leader key 로 "누가 Primary 인가" 를 하나로 정한다. 앱은 VIP 하나만 안다.

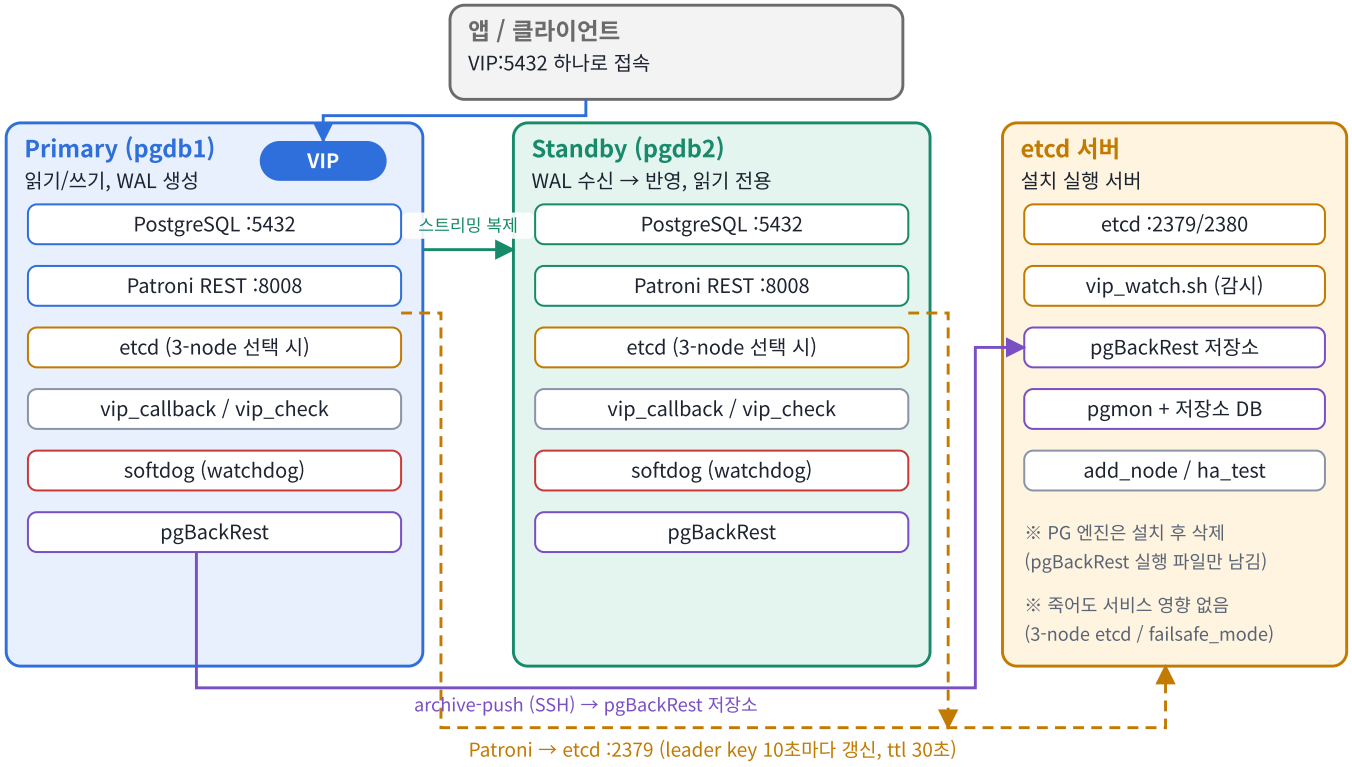
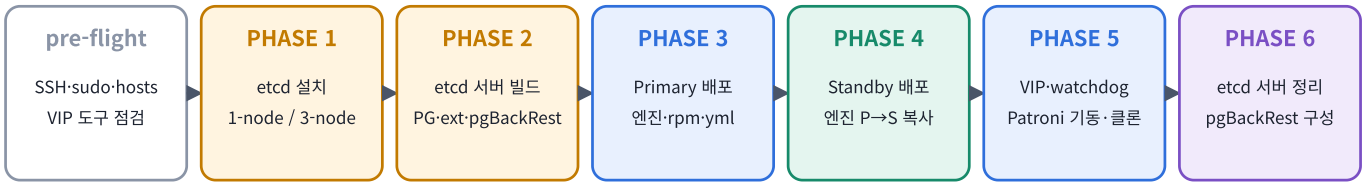


그림 4. HA 런타임 구성 (3-node etcd 를 고르면 DB 노드에도 etcd)

서버	설치 / 실행되는 것	죽으면
Primary	PostgreSQL, Patroni(systemd), VIP 스크립트(callback + cron), softdog, pgBackRest(archive-push), (3-node) etcd	약 30~40초 뒤 Standby 자동 승격 + VIP 이동
Standby	Primary 와 동일 (Patroni 가 pg_basebackup 으로 클론)	복제만 끊김, 서비스 계속. 살아나면 자동 재합류
etcd 서버	etcd, vip_watch(감시·알림), pgBackRest 저장소 + 백업 크론, pgmon (+ 저장소 DB single)	3-node etcd 면 영향 없음 / 1-node 면 failsafe_mode 로 Primary 유지(그동안 failover 불가). 감시·백업만 멈춤

5. HA 설치 흐름

etcd 서버에서 `bash postgresql_install.sh` → ha. 빌드는 etcd 서버에서 한 번만 하고 SSH/scp 로 배포한다.



- 하나라도 실패하면 조치 방법을 출력하고 멈춤 (pre-flight). 설치 로그: /pgdata/log/pg_install.log, ha_setup.log
- Primary 는 REST /primary=200, Standby 는 /replica=200 이 될 때까지 기다린 뒤 다음 단계로 (systemctl active 만 보지 않음)
- etcd 서버에 single PG(pgmon 저장소)가 있으면 PHASE 2 전에 엔진을 옮겨두고 PHASE 6(또는 실패 시)에 되돌린 뒤 재기동

그림 5. 설치 단계 (PHASE)

6. 장애 시 동작

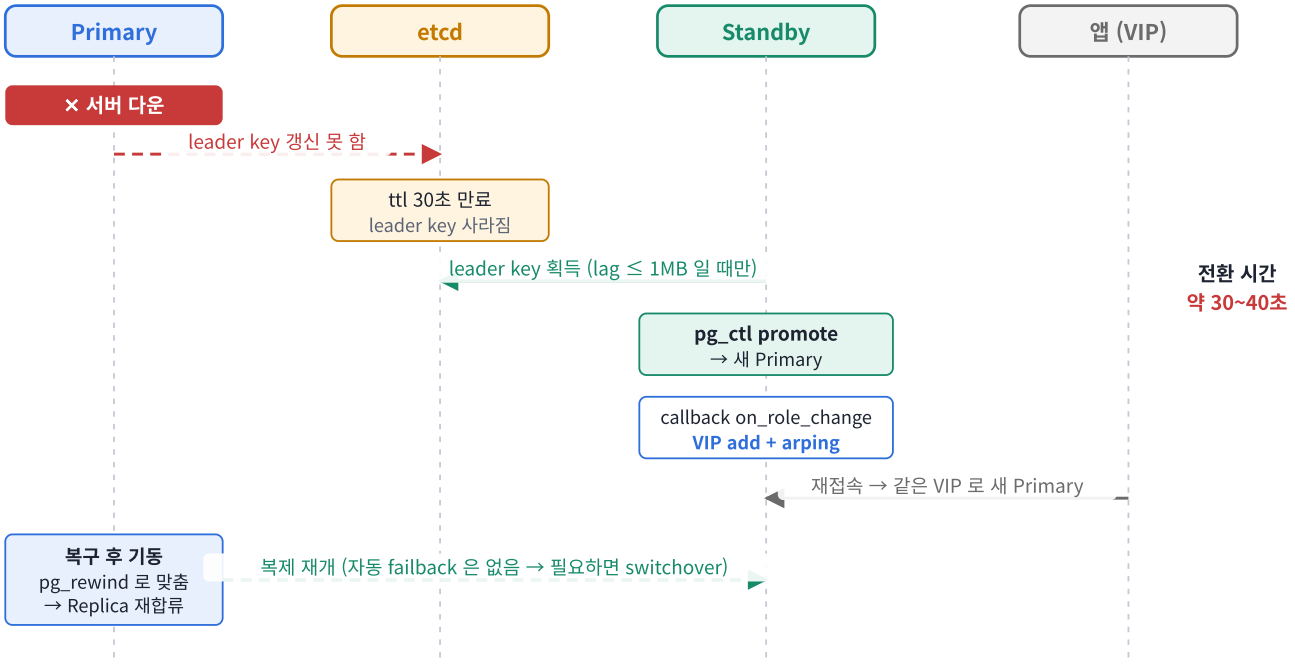


그림 6. Primary 서버 다운 → 자동 failover

상황	무엇이 막아주나	결과
Primary 서버 다운	Patroni leader key 만료 → Standby 승격	약 30~40초 후 서비스 재개 (VIP 이동)
Primary 네트워크만 단절	Primary Patroni 가 etcd 접근 불가 → **스스로 demote** + VIP 제거	반대편 승격. 구 Primary 는 쓰기 불가 상태라 P-P 안 생김
Primary freeze (커널 hang, VM 정지, Patroni 멈춤)	**watchdog(softdog)** — Patroni 가 25초 안에 못 두드리면 커널이 노드 리부트	멈췄던 노드가 깨어나 Primary 로 남는 것 (split-brain) 방지
etcd 전체 장애	**failsafe_mode** — Primary 가 모든 멤버와 REST(8008) 통신되면 유지	쓰기 계속. 그동안 failover 불가 (끄면 Primary 도 demote → 쓰기 중단)
failover 시 데이터 유실	maximum_lag_on_failover 1MB / **synchronous_mode**(선택)	sync 면 유실 0, 대신 커밋 지연 증가. Standby 장애 시 자동 비동기 전환
Standby 다운	—	서비스 영향 없음. 살아나면 자동 재합류
callback 누락 (VIP 안 붙음)	vip_check (각 노드 20초) → vip_watch (etcd 서버)	20~40초 안에 보정 + 알림

7. 안전장치 (split-brain / 유실 방지)

Patroni 만으로는 "멈춘 Primary 가 나중에 깨어나는" 경우를 막지 못한다. 그래서 세 가지를 설치 단계에서 같이 켠다 (입력으로 선택).



그림 7. watchdog · failsafe_mode · synchronous_mode

8. VIP 3중 관리

Primary 를 따라 이동하는 가상 IP. 같은 L2 서브넷 전제 (클라우드는 LB / DNS). 즉시 반응(callback) → 놓친 것 보정(vip_check) → 바깥에서 감시(vip_watch) 순으로 겹쳐 둔다.

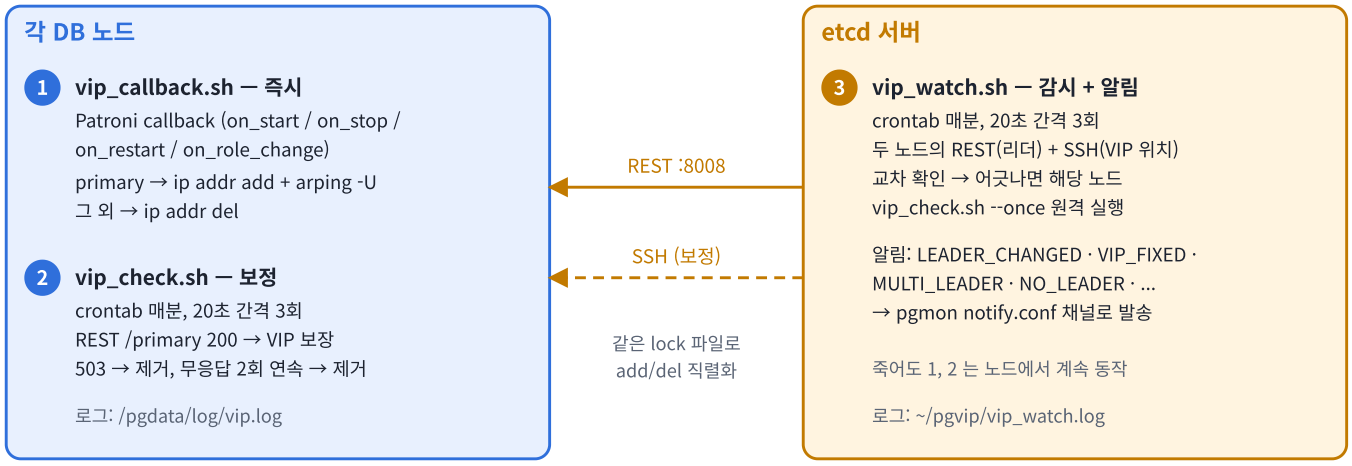


그림 8. VIP 관리 구조

9. 백업 — basebackup vs pgBackRest

설치 때 `백업 선택 (pgbackrest / basebackup)` → Archive → Backup 순으로 묻는다. HA 에서 pgBackRest 를 고르면 **etcd 서버가 저장소**가 되어 공유 스토리지 없이도 failover 와 무관하게 WAL 이 이어진다.

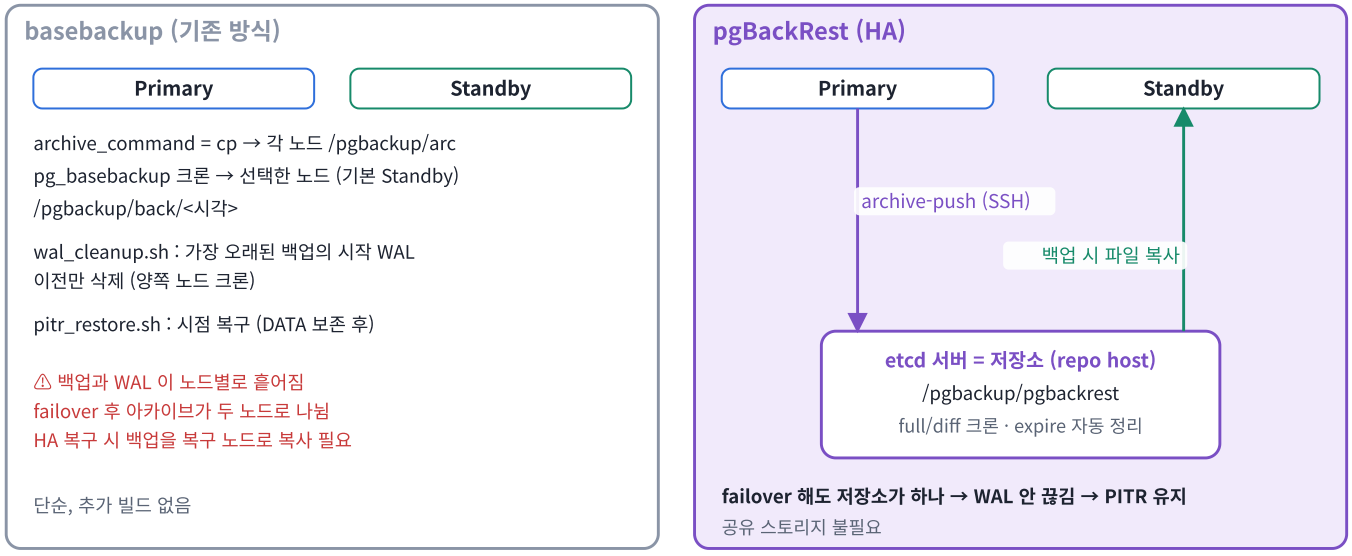


그림 9. 두 방식 비교 (HA 기준)

	basebackup	pgBackRest
백업 종류	매번 전체 (tar.gz)	full / diff / incr, 압축(lz4), 병렬
보관 정리	BACKUP_KEEP 개수 + wal_cleanup.sh	repo1-retention-full (expire 자동)
기본 크론	매일 02:00	일요일 02:00 full + 월~토 diff
복구	pitr_restore.sh	pgbr_restore.sh (--target-time / --set / --delta)
HA 저장 위치	각 DB 노드 (백업은 선택한 노드)	etcd 서버 한 곳 (backup-standby=prefer)
추가 준비	없음	설치 시 소스 빌드(meson), DB 노드 → etcd 서버 SSH 키 (자동 등록)

HA 의 archive_mode / archive_command 는 patroni.yml DCS 파라미터에 들어가 첫 기동부터 동작한다. pgBackRest 면 Standby 의 restore_command 도 저장소의 archive-get.

10. 노드 추가 / DR

etcd 서버에서 `bash add_node.sh`. 새 서버는 계정·sudo·디렉토리·/etc/hosts·SSH 키만 준비하고 PG 는 설치하지 않는다.

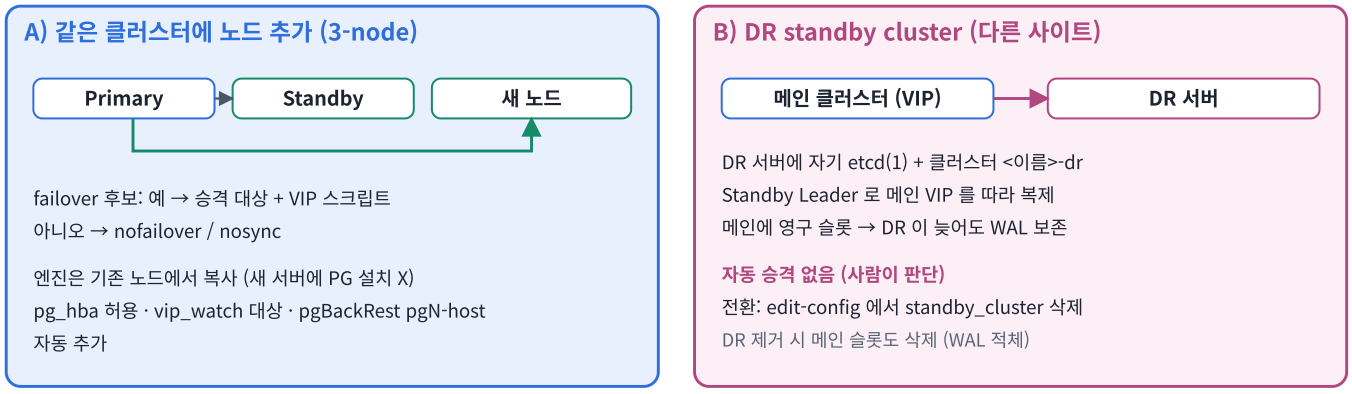


그림 10. add_node.sh 두 가지 방식

11. 모니터링 — pgmon

psql + bash 만으로 동작하는 터미널 모니터링. 수집은 DB 별 collector 프로세스가 하고(root 불필요), 데이터는 저장소 DB `pgmon` 에 쌓는다.

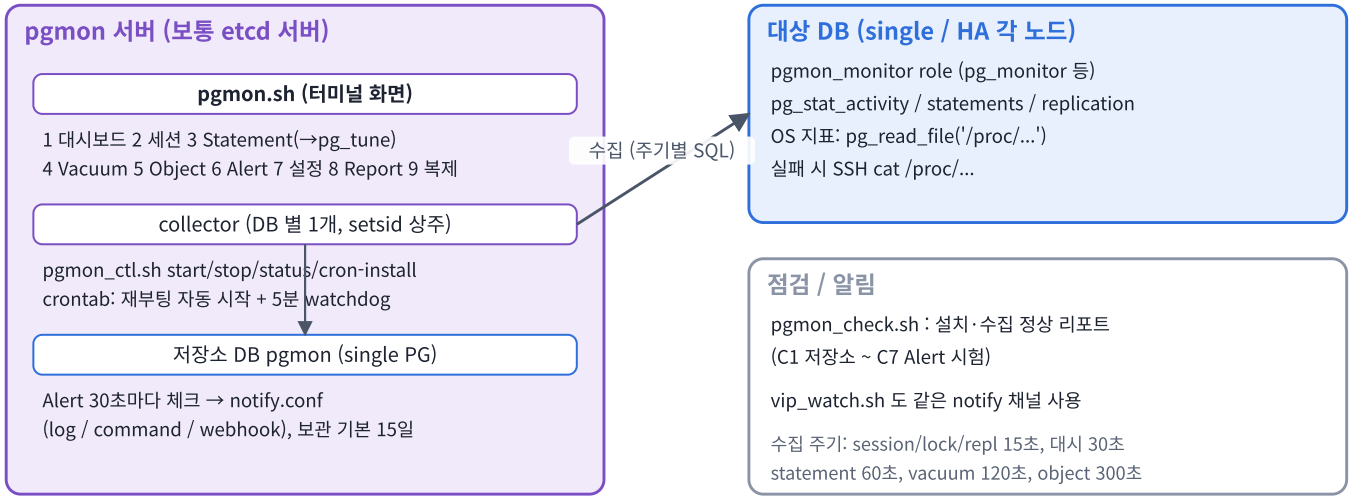
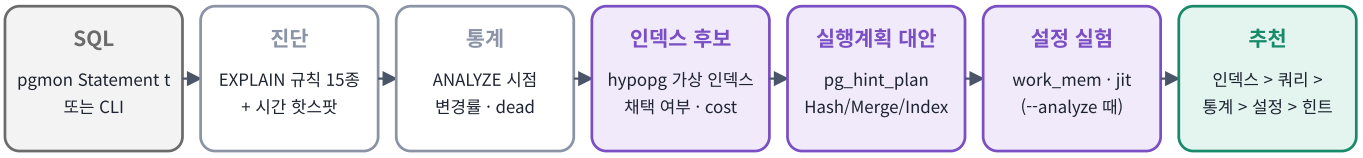


그림 11. pgmon 구조

12. 쿼리 튜닝 — pg_tune

AI 없이 규칙 진단 + 실제 검증. pgmon Statement 화면에서 `t` → 번호로 바로 실행하거나 CLI 로 쓴다.



- python3 표준 라이브러리 + psql 만 사용 (pip 설치 없음). 기본은 추정 플랜, --analyze 는 실제 실행 (DML 은 ROLLBACK)
- 보라색 단계는 '실제로 해봐서' 검증 — 그냥 규칙만 보고 추천하지 않음. 해당 extension 이 없으면 그 단계만 '검증 불가'

그림 12. pg_tune tune 파이프라인

진단 규칙 (예)	조치 방향
대부분 버리는 Seq Scan / 인덱스 후 Filter 로 대부분 버림	단일 · 복합 인덱스 후보 (hypopg 로 검증)
LIMIT 때문에 전체 정렬 (Gather Merge 포함)	(= 컬럼 + 정렬 컬럼) 복합 인덱스
Nested Loop 수천 회 반복 / 상관 서브쿼리	조인 키 인덱스, Hash/Merge Join 실험, JOIN/EXISTS 재작성
컬럼에 함수 · 캐스트 · LIKE '%..'	범위 조건 재작성, 표현식 인덱스, pg_trgm (해당 컬럼은 인덱스 후보에서 제외)
추정 vs 실제 행수 10배+	ANALYZE / CREATE STATISTICS, Rows 힌트 실험
Sort/Hash 디스크 spill, lossy bitmap, JIT 과다	work_mem · jit=off 실험 (--analyze)

13. 포트 · 디렉토리

방화벽 허용 포트

출발 → 도착	포트	용도
앱 → VIP / Primary / Standby	**5432**	DB 접속 (입력한 포트)
Primary ↔ Standby	**5432**	복제, pg_rewind
DB 노드 → etcd 노드	**2379**	Patroni → etcd
etcd 노드 ↔ etcd 노드	**2380**	etcd 피어 (3-node)
etcd 서버 → DB 노드	**8008**	Patroni REST (vip_watch, 설치 확인)
Primary ↔ Standby	**8008**	Patroni REST (failsafe_mode)
etcd 서버 → DB 노드	**22**	SSH (설치, vip_watch, pgBackRest 백업)
DB 노드 → etcd 서버	**22**	SSH (pgBackRest archive-push)
Primary ↔ Standby	**22**	SSH (설치)
pgmon 서버 → DB 노드	**5432 / 22**	수집 / OS 지표 SSH fallback

설치 후 디렉토리 (기본값)

경로 (기본값)	내용	위치
`/postgres/app`	엔진 (bin, lib, share), etc/configure_options.txt	DB 노드
`/postgres/app/etc/patroni`	patroni.yml, vip_callback.sh, vip_check.sh	DB 노드 (HA)
`/postgres/app/etc/backup`	basebackup/, wal_cleanup.sh, pitr_restore.sh	DB 노드
`/postgres/app/etc/backup/pgbackrest`	bin, lib(libpq), conf, log, pgbr_restore.sh	모든 서버
`/pgdata/data`	데이터 (HA: postgresql.conf 는 Patroni 관리, 튜닝값은 postgresql.base.conf)	DB 노드
`/pgdata/log`	PG 로그, pg_install.log, ha_setup.log, vip.log, wal_cleanup.log	DB 노드
`/pgbackup/arc, /back, /dump`	basebackup 아카이브 / 백업 / dump	DB 노드
`/pgbackup/pgbackrest`	pgBackRest 저장소	저장소 서버
`~/.postgresrc`	PGHOME, PGDATA, PATH, alias (pgstart, pglog ...)	DB 노드
`~/pgvip/`	vip_watch.sh, vip_watch.log	etcd 서버
`/etc/etcd/etcd.conf, /var/lib/etcd`	etcd 설정 / 데이터 (systemd: etcd)	etcd 노드
`pgmon/conf/`	pgmon.conf(600), notify.conf, collector 로그, pgtune_log/	pgmon 서버

보안 기본값 — 운영 전 반드시 변경

- single: initdb 기본이라 **로컬 접속 trust** → postgres 패스워드 설정, pg_hba 조정
- HA: patroni.yml superuser 패스워드 `postgres`, 노드 IP 기준 **/24 trust** → 변경 / 범위 축소
- pgmon role 패스워드 = role 이름 → 변경
- 설치 계정 sudo NOPASSWD: ALL → 운영 시 systemctl patroni, ip, arping 정도로 축소 가능 (운영가이드 참고)